

GeoTrace Forensics

Presentation Cheat Sheet

Demo script, talking points, and Q&A preparation

Team: Selim • Lojain • Moussa

Total demo: 12 min • Q&A: 5 min

Before the demo (setup checklist)

Complete these checks 15 minutes before presenting:

1. Laptop fully charged and plugged into power.
2. Python virtual environment activated.
3. Test images already loaded in a fresh case named 'Final Demo Case'.
4. Image set: 3 GPS-tagged images (DSCN0010, DSCN0012, DSCN0029) + 2 tampered images.
5. Run app once before the demo — confirm everything works.
6. Close all extra browser tabs and notifications.
7. Open the project folder in File Explorer (for the blockchain tamper test).
8. Have the test_images folder ready in case you need to re-import.

Pro tip: keep a SECOND case ready called 'Tamper Test' with only 2 images. After Lojain shows Chain Intact, she modifies that case's blockchain.json (not the demo case) to prove tampering detection.

Demo timeline overview

Time	Speaker & Topic
0:00 - 1:00	Selim: Introduction + tool overview
1:00 - 4:00	Selim: GUI walkthrough + EXIF + SHA-256 + AI demo
4:00 - 8:00	Moussa: Database + GPS decoder + Map + Timeline
8:00 - 12:00	Lojain: Anomalies + Blockchain + Tamper Test + PDF
12:00 - 17:00	Q&A — all members answer

Each member transitions smoothly. Practice the handoff: 'Now Moussa will show you how the geolocation works...'

SELIM — Segment 1 (4 min)

Role: Core Engine + AI Detection

Files Selim owns

- `main.py` — application entry point
- `ui/main_window.py` — the GUI window with all tabs
- `modules/exif_extractor.py` — reads metadata from images
- `modules/hash_calculator.py` — SHA-256 file fingerprinting
- `modules/ai_detector.py` — BONUS: detects manipulated images

Selim's demo script

Opening (30 seconds)

Selim: "Good morning. We are presenting GeoTrace Forensics, a digital forensic tool for investigating image evidence. Modern photos contain hidden metadata — GPS coordinates, timestamps, camera info. Our tool extracts this data, detects manipulation, and maintains a tamper-proof chain of custody."

Show the GUI (30 seconds)

[ACTION] Run `'python main.py'` from terminal.

Selim: "This is the main interface, built in PyQt5. You can see the El Sewedy logo, case management section, action buttons, evidence table, and detail tabs."

Create case + import images (1 minute)

[ACTION] Click 'Open Existing Case' dropdown → select 'Final Demo Case'.

Selim: "I already have a case loaded with five images. When images are added, the tool automatically calculates a SHA-256 hash for integrity, extracts all EXIF metadata, runs anomaly detection, and adds the file to the blockchain evidence log."

[ACTION] Click the first image in the table (`DSCN0010.jpg`).

[ACTION] Click the 'Metadata' tab.

Selim: "Here you can see camera make is Nikon, capture date is 2008, GPS coordinates point to Italy. The `exif_extractor.py` module reads both Pillow and ExifRead formats to support all image types."

AI Analysis demo (2 minutes — the highlight)

[ACTION] Click on a tampered image in the table.

[ACTION] Click the 'AI Analysis' tab.

[ACTION] Click 'Run AI Analysis'.

Selim: "Our bonus feature is AI-based manipulation detection. It uses three techniques: Error Level Analysis or ELA, which re-saves the image at low quality and detects regions that compress differently — these are the edited parts. Second is noise consistency — authentic cameras have uniform sensor noise across the image. Third is metadata cross-validation."

[ACTION] Point at the ELA image and verdict.

Selim: *"You can see the verdict: Probably Manipulated. The ELA image shows bright noise patterns where the manipulation happened. The metadata findings confirm Adobe Photoshop CS was used. All three signals combined into a final confidence score."*

Hand off to Moussa

Selim: *"Now Moussa will explain how we store all this data and visualize geolocation."*

MOUSSA — Segment 2 (4 min)

Role: Database + Geolocation

Files Moussa owns

- modules/db_manager.py — SQLite database manager
- database/schema.sql — table structure
- modules/gps_decoder.py — GPS coordinate decoding
- modules/map_generator.py — interactive Folium map
- modules/timeline_generator.py — chronological timeline + speed detection

Moussa's demo script

Database explanation (1 minute)

Moussa: "Thank you Selim. All evidence is stored in a SQLite database with four linked tables: cases hold investigations, images store file info and hashes, metadata holds extracted EXIF data, and anomalies records suspicious findings. The schema uses foreign keys so deleting a case automatically cleans up related data."

GPS decoding explanation (1 minute)

[ACTION] Click an image with GPS data, click 'Metadata' tab.

Moussa: "GPS in EXIF is stored as degrees, minutes, seconds — DMS format. For example 43 degrees 28 minutes 2 seconds. To plot on a map we need decimal degrees like 43.467. The gps_decoder module converts this and validates the coordinates are within Earth's range — minus 90 to 90 latitude, minus 180 to 180 longitude."

Map demo (2 minutes — the highlight)

[ACTION] Click 'Generate Map' button.

[ACTION] Wait for browser to open.

Moussa: "Our map uses Folium, which is Python's wrapper for Leaflet.js. The pins show locations of GPS-tagged images. The dashed line connects them chronologically — this is the suspect's movement path. The case summary in the top-right shows totals."

[ACTION] Click on a green clustered pin to zoom in.

Moussa: "Pins are color-coded by status: green for clean, orange for anomalies, red for integrity issues, dark red for suspicious movement — that's calculated using the Haversine formula. If a person moves more than 300 km/h between photos, that's physically impossible and gets flagged."

[ACTION] Use the time slider at the bottom to animate.

Moussa: "The timeline slider lets investigators replay events in chronological order — useful for building a suspect's movement story."

Hand off to Lojain

Moussa: "Now Lojain will demonstrate how we detect manipulation in metadata and how we protect the entire evidence chain using blockchain."

LOJAIN — Segment 3 (4 min)

Role: Anomalies + Blockchain + Reporting

Files Lojain owns

- modules/anomaly_detector.py — flags suspicious EXIF patterns
- modules/blockchain_log.py — BONUS: tamper-proof evidence chain
- modules/report_generator.py — PDF forensic report
- modules/export_manager.py — Excel and JSON exports
- ui/style.qss — GUI styling

Lojain's demo script

Anomalies explanation (1 minute)

[ACTION] Click a tampered image, click 'Anomalies' tab.

Lojain: *"Thank you Moussa. The anomaly detector runs nine forensic checks on every image. For this tampered image, it found editing software detected — Adobe Photoshop — and missing camera information, which is a sign of re-exporting. Each anomaly has a severity level: high, medium, or low. Investigators use these to decide which images need deeper analysis."*

Blockchain demo (2 minutes — the highlight)

[ACTION] Click the 'Blockchain' tab.

[ACTION] Click 'Verify Chain'.

Lojain: *"Our second bonus is blockchain chain of custody. Every image added to the case becomes a block. Each block contains the file name, SHA-256 hash, investigator, timestamp, and a cryptographic link to the previous block. You can see all five blocks here, all verified — Chain Intact."*

Live tamper test

Lojain: *"Let me demonstrate why this matters. I'll prove that if anyone modifies the evidence log, the system detects it instantly."*

[ACTION] Open File Explorer → navigate to output/case_X/blockchain.json

[ACTION] Open the file in Notepad.

[ACTION] Change one character in any file name (e.g., DSCN0010 → DSCN0011).

[ACTION] Save the file.

[ACTION] Go back to the app, click 'Verify Chain' again.

Lojain: *"Now you can see the chain is broken. The tool identified the exact block that was tampered with, and shows that all subsequent blocks lost their cryptographic link. This is the same principle used in cryptocurrencies, but applied to forensic evidence — proving in court that no one has altered our evidence log."*

PDF Report demo (1 minute)

[ACTION] Go back to the main case.

[ACTION] Click 'Generate PDF Report'.

[ACTION] Open the PDF.

Lojain: *"Finally, this is the court-ready report. It includes the case summary, evidence chain with all SHA-256 hashes, timeline findings, correlations between images, and the complete anomaly register. Reports can also be exported to Excel or JSON for use in other forensic tools."*

Closing

Lojain: *"That concludes our demonstration. We are happy to take your questions."*

Q&A — top 12 expected questions

Examiners typically ask about: technical depth, design choices, limitations, and alternatives. Each question is assigned to the member who owns that area.

Technical questions

Q1. Why did you use SHA-256 instead of MD5 or SHA-1?

Answer (Selim):

MD5 and SHA-1 are cryptographically broken — collision attacks have been demonstrated. SHA-256 is the current industry standard for forensic integrity. NIST and the FBI recommend it. It produces a 256-bit output that makes accidental or intentional collisions effectively impossible.

Q2. How does Error Level Analysis actually detect manipulation?

Answer (Selim):

JPEG compression introduces a known level of error in every region of an image. When you re-save the entire image at a known quality, untouched regions show consistent compression error, while edited regions — which were already compressed once before being pasted in — show a different error pattern. The difference is visualized as brightness: edited areas appear brighter.

Q3. Why use SQLite instead of MySQL or PostgreSQL?

Answer (Moussa):

SQLite is a serverless, single-file database — no installation, no configuration, no network. For a forensic tool that runs on an investigator's laptop, this is ideal. The whole case can be backed up by copying one file. For larger multi-user deployments we would migrate to PostgreSQL, but SQLite handles single-investigator cases perfectly.

Q4. How does GPS decoding work mathematically?

Answer (Moussa):

EXIF stores GPS in degrees-minutes-seconds format with reference letters N/S/E/W. The decimal degree formula is: $\text{degrees} + (\text{minutes} / 60) + (\text{seconds} / 3600)$. Then if the reference is South or West, we negate the result. We also validate the result is within Earth's range and not at 0,0, which is a placeholder value.

Q5. Is your blockchain a real blockchain?

Answer (Lojain):

It's a private, single-writer blockchain — not a distributed public blockchain like Bitcoin. We don't need consensus mechanisms or mining because there's only one investigator. What we use is the core cryptographic principle: each block contains a hash of the previous block, so modifying any block breaks all subsequent blocks. This proves integrity, which is the actual forensic requirement.

Q6. What happens if I lose the blockchain file?

Answer (Lojain):

The evidence chain would be lost. That's why our recommended workflow is to back up the blockchain.json file separately from the case database — for example to write-once media or a cloud backup. This is standard chain-of-custody practice — physical evidence logs are also backed up.

Design questions

Q7. Why PyQt5 instead of a web interface?

Answer (Selim):

Forensic investigators often work on isolated, offline machines for security reasons. A desktop app guarantees no network calls and no data leaving the workstation. PyQt5 is cross-platform and provides native OS-level performance. A web version would require running a local server, adding complexity and attack surface.

Q8. What image formats do you support?

Answer (Selim):

JPEG, PNG, TIFF, BMP, WEBP, HEIC, HEIF, and GIF. We use Pillow combined with ExifRead because they support different format families. ELA works best on JPEG due to its compression model; PNG support is limited because it's a lossless format.

Q9. Why Folium instead of Google Maps or Mapbox?

Answer (Moussa):

Folium uses OpenStreetMap which is free and doesn't require an API key. For a forensic tool we cannot have the investigator's case data sent to a third party for map rendering. Folium generates HTML that runs offline once loaded, preserving evidence confidentiality.

Limitations and improvements

Q10. What are the limitations of your AI detector?

Answer (Selim):

Our detector is heuristic-based, not deep learning. Limitations: ELA loses accuracy on PNG files; very high JPEG compression can cause false positives because the entire image looks 'edited'; AI-generated images from new models like Stable Diffusion are not specifically targeted. A future improvement is integrating a CNN trained on the CASIA dataset.

Q11. Can your tool be fooled by someone who knows about it?

Answer (Lojain):

Yes, if they have access to both the evidence files AND the blockchain log AND understand cryptography, they could potentially rebuild a valid chain — but only by replacing every block from the modified point forward. In practice this is detectable because timestamps would be inconsistent, and writing to write-once media prevents it entirely. Forensic tools rely on physical and procedural safeguards combined with cryptographic ones.

Q12. What would you add if you had more time?

Answer (any member):

Three things: First, a deep-learning manipulation detector trained on the CASIA dataset, replacing our heuristic approach. Second, reverse image search to find if photos appear online elsewhere. Third, multi-investigator collaboration with role-based access control — useful for team investigations where different people add evidence.

Emergency tips — if something goes wrong

If the app crashes during demo

- Stay calm. Say: 'Let me restart the application' — this is normal.
- Close the app, run `python main.py` again.
- Continue from where you left off.

If a question is unexpected

- Pause for 2 seconds. Don't rush an answer.
- Say: 'That's a great question. Based on our research...'
- If you don't know, say: 'We didn't explore that — but here's how I would approach it...'
- Never invent technical details. It's better to say 'I'd need to check that' than to be wrong.

If the examiner challenges your design

- Acknowledge their point: 'That's a valid concern.'
- Explain your reasoning: 'We chose X because...'
- Show flexibility: 'In a production version we would consider Y.'
- Never get defensive — examiners are testing how you think, not just what you built.

If the map or PDF takes too long

- Talk while it loads: 'While this generates, let me explain...'
- Have a backup screenshot or pre-generated PDF ready as fallback.

If you forget a technical term

- Describe what it does instead of naming it: 'The thing that converts coordinates...'
- Your teammate can help — they can chime in with the term.

Final pre-demo checklist (5 min before)

9. All 3 members present and ready.
10. Each member knows their files.
11. App tested 30 minutes ago — confirmed working.
12. Demo case loaded with images.
13. Backup tamper test case ready.
14. Browser tabs closed.
15. Notifications disabled.
16. Water nearby.
17. Deep breath. You've got this.